NFAサイト AI検索 設計書

Oct 8, 2026 · @Yamaguchi

第1部 概要（クライアント向け）

1. 目的と背景

WordPressの標準検索では届かない記事やPDFまで、利用者の言葉から探せるようにします。サイト全体はWordPressのテーマで構築し、検索部分だけに独自プラグインでAIを組み込みます。

長野県サッカー協会の情報は、委員会ごとに分かれて発信されています。利用者は「だれの・何の情報か」で探したいのに、標準検索は次のものに届きません。

届かないもの	例
言い換え	「小学生」と書いても「4種」の記事が出ない／「笛を吹きたい」から「審判」に届かない
PDFの中身	大会要項・様式の本文は検索対象外
分類の組み合わせ	「小学生 × 試合結果」のような条件の掛け合わせ
意図の推測	0件のとき、近い情報を提案できない

標準検索はタイトルと本文の文字列一致のみのため、上の課題は「発信の構造」から「探索の構造」へ切り替えることで解決します。

2. 実現する体験

探索画面は、ワイヤー（`NFA_wireframes_site/search.html`）の画面と擬似対話をそのまま本物のデータで動かすものです。左に「条件（構造）」、右に「会話（経過）」を常に並べ、いまの条件と探していたことの両方が見えるようにします。

1. 自由な言葉で入力できる（例：「小学生の大会っていつ？」「審判の資格更新はどうすれば？」）
2. AIが言葉を「対象・目的・地域」の3軸の条件に読み替え、「〜のこと、〜したいですね。N件の情報があります。」と返す
3. 結果を新しい順に5件表示し、6件以上なら「この条件のN件をすべて見る」で一覧へ移る
4. 次に絞れる軸をひとつだけ提案する（「さらに絞り込むなら（地域）」）
5. 左の条件パネルには選択肢ごとの件数を出し、0件になる選択肢は押せなくする

6. 0件になったら「ひとつ条件を外すと、見つかるかもしれません」と外す条件を提案し、読み取れないときは「よく選ばれている条件」を示す

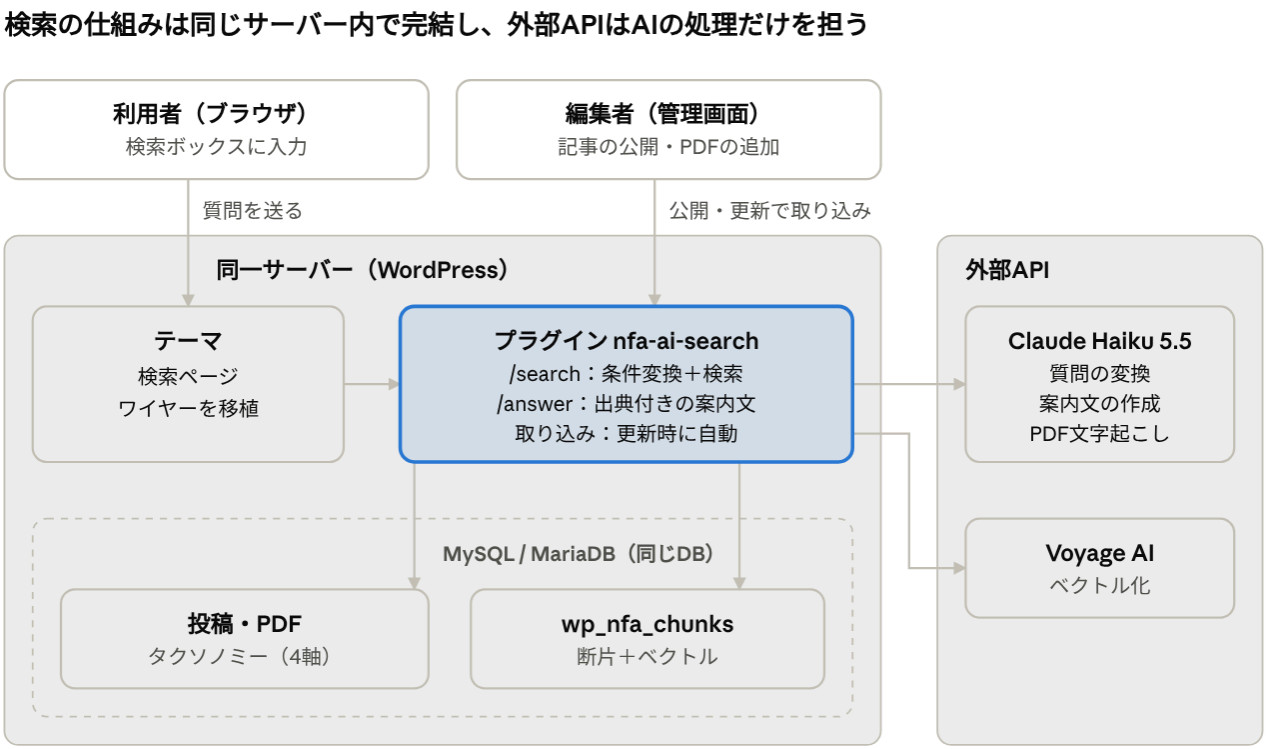
条件は軸ごとにひとつで、ANDで累積します。足すほど件数は減り、増えることはありません。

正確さのルール：AIの案内文は、検索で取り出した協会の掲載ページだけを根拠に作ります。必ず出典ページへのリンクを付け、出典ページ側にも「AIの案内はこのページを根拠にしています」と表示します。掲載ページにないことをAIが答えることはありません。

個人情報の扱い：お問い合わせの内容、会員・ユーザー情報、下書きや非公開の記事は、検索の対象にもAIへの送信対象にもなりません。公開済みの記事・固定ページ・PDFのうち、検索してよいと決めた種類だけを取り込みます（9章）。

3. 全体構成

WordPressのテーマと独自プラグイン `nfa-ai-search` を同じサーバーに置き、外部に出すのはAIの処理だけです。別のサーバーは必要ありません。



全体構成 · 同一サーバー＋外部API 2つ

利用者の質問はプラグインが受け取り、Claudeで条件に変換してから、同じデータベースの投稿とベクトルを検索します。APIキーはサーバー内にあり、ブラウザには出ません。

4. 費用

AIの利用料は、標準構成なら月数百〜1万円程度です。サーバーはWordPressと同じものを使うため、追加のサーバー代は原則かかりません。費用の大部分は、AIの利用料より開発費と運用の手間です。

前提：1ドル＝150円。Claude Haiku 5.5 は100万トークンあたり入力$0.10／出力$0.50、Claude Sonnet 5.5 は入力$2／出力$10。埋め込みAPIの単価は概算で、採用時に確認します。

1回の検索あたり

処理	使うモデル	1回あたり
① 質問を検索条件に変換	Haiku	約0.05円
② 質問のベクトル化・類似検索	Voyage AI など	ほぼ0円
③ 出典付きの案内文	Haiku	約0.1〜0.2円
③ 同上（文章の質を上げる場合）	Sonnet	約2〜5円

月額の幅（検索回数別）

月間検索数	最小（①のみ・Haiku）	標準（①〜③・Haiku）	高品質（③をSonnet）
1,000回	約50円	約200円	約2,000〜5,000円
10,000回	約500円	約2,000円	約2〜5万円
50,000回	約2,500円	約1万円	約10〜25万円

同じ質問の結果を一時保存して使い回すと、上の額から3〜5割ほど下がる見込みです。Anthropicの管理画面で毎月の利用上限を設定し、上限を超えたらAIを使わない検索に自動で切り替えます。

初回だけかかる費用（記事5,000件・PDF 300件）

内容	費用
記事・PDFのベクトル化	数十〜数百円
スキャンPDFの文字起こし（Haiku）	数百円
既存記事への3軸の自動分類（Batch API）	数百円

その後は、記事更新のたびに差分だけを処理するため、ほぼ0円です。

5. 進め方

3段階に分け、段階ごとに効果を確かめながら進めます。段階1だけでも、ワイヤーの探索画面が本物のデータで動きます（開発環境で実装済み）。

段階	内容	できるようになること	AI費用
1	タクソノミー（3軸）と辞書の整備、既存記事の自動分類、「質問→検索条件」の変換	言い換え・条件の組み合わせで探せる	小
2	記事本文・PDFのベクトル検索を追加	PDFの中身や意味の近い記事も出る	中
3	出典付きの案内文、0件時の言い換え提案	対話しながら探せる	中

各段階の日程は、サーバー契約とテーマ制作のスケジュールが決まってから組みます。

6. 決定事項と確認待ち事項

構成は決まっており、残る確認事項は4つです。いずれも段階1の着手を妨げません。

決定事項

- サイト全体をWordPressのテーマで構築し、記事もWordPressで公開する。探索画面はワイヤー search.html の形、分類は対象・目的・地域の3軸
- AI検索は同じサーバー上の独自プラグインで実現する（別サーバー不要）
- PHPは最新版、SSHが使える、外部APIへの接続は可能な見込み
- データベースはMySQLかMariaDB。MySQL 8.0 になる可能性があり、その場合も同じサーバー内で実現する
- PDFの中身も検索対象にする（お問い合わせ内容など個人情報を含むデータは対象外）
- 規模の前提：記事約5,000件（今後増加）、PDF約300件

確認待ち事項

- [ ] サーバー契約：データベースの種類とバージョン（MySQLなら 8.4 LTS を選べるか）
- [ ] 月間の検索数の見込み（現サイトのアクセス数から推定）
- [ ] PDFの置き場所（メディアライブラリか、外部リンクか）と実数
- [ ] AI費用の月額上限

第2部 技術仕様（開発者・サーバー担当向け）

7. サーバー要件

データベースは MySQL 8.0 系と MariaDB のどちらでも実現できます。MariaDB 11.8 以降なら標準のベクトル検索を使い、MySQL 8.0 ならPHPで類似度を計算する方式にします。

項目	要件	状況
PHP	最新版	決定
データベース	MySQL 8.0 系（8.4 LTS 推奨）または MariaDB。MySQL の場合は下の「MySQL 8.0 の場合」の方式	MySQL 8.0 の可能性あり
外向きHTTPS通信	`api.anthropic.com` と埋め込みAPIに接続できること	サーバー上の `curl` で確認
`max_execution_time`	30秒以上	確認
`memory_limit`	256MB以上	確認
cron	サーバーのcronで `wp cron event run` を定期実行（WP-Cronはアクセス依存のため）	確認
SSH＋WP-CLI	初回の一括取り込みに使用	決定
出力のバッファリング	回答文を逐次表示する場合のみ確認（なくても動く設計）	任意

MySQL 8.0 の場合

MySQL 8.0 にはベクトル型がないため、ベクトルはバイナリ列（BLOB）に保存し、類似度の計算はPHPで行います。この規模なら、次の工夫で同じサーバー内に収まります。

工夫	内容
ベクトルを小さくする	次元を512に減らし、1要素1バイト（int8）で保存。3万件で約15MB（埋め込みモデルが対応していることが前提）
候補を先に絞る	タクソノミー（3軸）の条件で絞ってから類似度を計算する
メモリに置く	ベクトルをAPCuなどのキャッシュに載せ、毎回DBから読まない

工夫	内容
日本語の全文検索	キーワード検索はMySQLのngram全文索引で強化する

類似検索の時間は、3万件で0.3〜0.8秒を見込みます。断片が5万件を超えるか、検索が1秒を超えるようになったら、外部のベクトルDB（Supabase、Cloudflare Vectorize など。無料枠〜月数千円）に切り替えます。そのため、ベクトルの保存先は差し替えできる作りにしておきます。

MySQL 8.0 は2026年4月にサポートが終了しています。後継の 8.4 LTS を選べないか、サーバー会社に確認してください。8.4 にもベクトル型はないため、上の方式は変わりません。

追加で必要な契約は、Anthropic API（利用上限を設定）と埋め込みAPI（Voyage AI など）の2つです。

8. データ設計

検索の土台は「3軸のタクソノミー」「言い換え辞書」「検索用の断片テーブル」の3つです。軸と選択肢はワイヤー（`search.html`）と `NFA_wireframes_site/README.md` の「3軸：対象・目的・地域」に合わせます。

カスタムタクソノミー（3軸＋管理用）

軸	スラッグ	選択肢	公開
対象（誰に向けた情報か）	`nfa_target`	小学生（4種）、中学生（3種）、高校生（2種）、大人・社会人（1種）、女子、シニア、フットサル、障がい者サッカー、審判、指導者	探索画面の絞り込み
目的（何をしたいか）	`nfa_purpose`	大会日程を知る、試合結果を見る、登録・手続き、講習会に参加、イベント・セミナーに参加、お知らせを読む	同上
地域（どの地区か）	`nfa_area`	北信、東信、中信、南信、県全域	同上
発信元（担当委員会）	`nfa_committee`	1種委員会、審判委員会、事務局など	管理用。公開側には出さない

審判・指導者は「立場」の軸を作らず、対象に含めます（1記事に複数の立場が絡むため）。「重要」「注意喚起」などは表示フラグなので、軸には入れません。

言い換え辞書

- 3軸の選択肢ごとに「入力文から読み取る言い換え（例：小学｜少年｜キッズ → 小学生（4種））」と「会話での言い回し（小学生（4種）のこと）」を持つ
- 初期値はワイヤー `search.html` の `AXES` / `LEX` / `utterance` を移植し、管理画面で編集できる
- 選択肢を増やすと、同名のタクソノミー用語も作られる
- AIへの指示文と、AIを使わない予備の読み取りの両方で使う
- 3軸に当てはまらない固有名詞（大会名など）は「キーワード」条件として扱う

検索用の断片テーブル `wp_nfa_chunks`

列	内容
`id`	主キー
`post_id`	元の投稿または添付ファイルのID
`source_type`	post / page / pdf
`chunk_index`	文書内での順番
`content`	断片の本文（数百字）
`embedding`	ベクトル（MySQL 8.0 は BLOB、MariaDB 11.8 なら `VECTOR` 型＋索引）
`url` / `title`	出典表示用
`content_hash`	変更がなければ再処理しないため
`updated_at`	鮮度管理用

規模は約2〜3万件から始まり、記事の増加に合わせて増えます。MariaDBの索引なら0.1秒以下、MySQL 8.0 の方式なら3万件で0.3〜0.8秒が目安です。

ベクトルの保存先と外部ベクトルDBへの切り替え

ベクトルの保存先は、共通の約束（インターフェース）を決めて差し替えられるようにします。最初は同じデータベース内で始め、規模が大きくなったら外部に移します。

```
interface NFA_Vector_Store {
    public function upsert( array $chunks ): void;          // [id, vector,
metadata]
    public function delete_by_post( int $post_id ): void;
    public function query( array $vector, int $top_k, array $filter ): array;
// [[id, score], ...]
}
// 実装：MySQL_Blob / MariaDB / Supabase / Vectorize
// wp-config.php の定数で切り替え：define( 'NFA_VECTOR_STORE', 'supabase' );
```

置き場所	持つもの
MySQL（WordPress）	記事、分類（3軸）、断片の本文、出典URL。こちらが正本
外部ベクトルDB	断片ID、ベクトル、絞り込み用の分類ID・日付だけ

外部側は「似ている断片のIDを返す索引」としてだけ使います。本文は送らず、壊れても `wp nfa reindex` でMySQLから作り直せます。

候補	向いている点	注意点
Supabase（pgvector、東京リージョン）※推奨	日本からの応答が速い。SQLで絞り込みを柔軟に書ける	無料プランは一定期間アクセスがないと停止する。本番は有料プランが安心
Cloudflare Vectorize	安価で、管理する項目が少ない	絞り込みに使う項目は事前に登録が必要

- 呼び出しは `wp_remote_post` で各サービスのREST APIを使う。管理用キーは `wp-config.php` に置き、ブラウザには出さない
- 外部ベクトルDBが3秒以内に応答しないときは、分類＋キーワードの検索だけで結果を返す
- 月1回、MySQL側と外部側の断片数を比べ、ずれていれば作り直す
- 料金と無料枠は採用時に確認する

9. 取り込み処理

記事の公開・更新とPDFのアップロードをきっかけに、裏で自動的に検索用データを更新します。編集者が特別な操作をする必要はありません。

きっかけ	フック／手段	処理
記事の公開・更新	`save_post`	その記事を再取り込みの待ち行列に入れる
記事の削除・非公開	`delete_post` / `transition_post_status`	断片を削除する
PDFのアップロード	`add_attachment`	PDFを待ち行列に入れる
初回・全件の作り直し	`wp nfa reindex`（WP-CLI）	SSHから時間制限なしで一括実行

待ち行列の1件ごとの処理：

1. 本文を取り出す（記事はブロックからテキスト化、PDFは smalot/pdfparser で抽出）
2. PDFから文字が取れない場合（スキャンPDF）は、Claude Haiku 5.5 にPDFを読ませて文字起こしする
3. `content_hash` が前回と同じなら終了（無駄なAPI費用を防ぐ）
4. 見出しごとに数百字ずつに分割する
5. 埋め込みAPIでまとめてベクトル化し、`wp_nfa_chunks` を入れ替える

待ち行列は Action Scheduler などで実装し、1回の処理を数秒に収めます。初回の一括取り込み（記事5,000件＋PDF 300件）は、数十分〜数時間を見込みます。

検索対象と除外ルール

検索対象は「許可リスト方式」で決めます。許可した種類の公開データだけを取り込むため、お問い合わせの送信内容や、あとから追加したプラグインのデータは、設定しなくても自動的に対象外になります。

区分	データ	条件
取り込む	投稿（`post`）	公開済みで、パスワード保護なし
取り込む	固定ページ（`page`）	同上。お問い合わせページや送信完了ページは個別に除外
取り込む	PDF（添付ファイル）	取り込み対象の公開記事に添付またはリンクされているもの
取り込む	カスタムフィールド	許可した項目（例：大会名、会場）のみ

区分	データ	条件
取り込まない	お問い合わせの送信内容	Contact Form 7 と関連アドオンのデータすべて（下の表）
取り込まない	下書き・非公開・予約・ゴミ箱・リビジョン	公開された時点で初めて取り込む
取り込まない	ユーザー情報、コメント、サイト設定、PDF以外の添付	—
取り込まない	記事を個別に除外したもの	投稿画面の「AI検索の対象外」チェック

許可リストは管理画面で設定します。新しい投稿タイプが増えても、誰かが許可するまでは対象になりません。

個人情報が漏れないよう、次の3か所で確認します。

1. **取り込み時**：許可リストと公開状態を確認してから取り込む。記事が非公開・削除になったら、断片をすぐに消す（外部ベクトルDB側も）
2. **検索時**：結果を返す直前に、元の記事がいまも公開中で許可対象かをもう一度確認する。プラグインは検索用テーブルと許可した投稿以外を読まない
3. **外部への送信時**：Claude、埋め込みAPI、外部ベクトルDBに送るのは、取り込み済みの公開データと利用者の質問文だけ。質問文中のメールアドレスと電話番号は伏せ字にしてから送る

Contact Form 7 の扱い

Contact Form 7 は送信内容をメールで送るだけで、それ自体はデータベースに残しません。関連するデータは、すべて検索対象外です。

データ	保存先	扱い
送信内容	保存しない（メール送信のみ）	対象外
添付ファイル	`wp-content/uploads/wpcf7_uploads` に一時保存し、送信後に自動削除	メディアライブラリに入らないため対象外
フォームの定義（項目、送信先アドレス、メール文面）	投稿タイプ `wpcf7_contact_form`	許可リストにないため対象外

データ	保存先	扱い
フォームを置いたページ	固定ページ	ページごと除外。他のページでも、取り込み時にショートコード（`[contact-form-7 …]`）は取り除く
Flamingo で保存した送信内容・連絡先	投稿タイプ `flamingo_inbound`、`flamingo_contact`、`flamingo_outbound`	対象外
CFDB7 など保存用アドオンのデータ	独自テーブル（例：`wp_db7_forms`）	プラグインから読まない

送信内容を管理画面に残したい場合は Flamingo を入れますが、その場合も検索対象には入りません。念のため、公開前のテストで「テスト送信した内容の言葉で検索しても0件になること」を確認します。

10. 検索API

APIは `/search`（結果を早く返す）と `/answer`（案内文を後から返す）の2本に分けます。待ち時間を短く見せ、各リクエストをPHPの制限時間内に収めるためです。

`GET /wp-json/nfa/v1/search?q=...`

1. 入力文を3軸の条件に読み替える（軸ごとにひとつ。まず辞書で、2軸以上取れなければ Claude Haiku 5.5 の構造化出力で）
   - 例：`{"target":"小学生（4種）","purpose":"大会日程を知る","area":"","keywords":[]}`
   - 指示文には辞書の選択肢と言い換えを含め、プロンプトキャッシュで費用を抑える
2. いまの条件に重ねる（同じ軸は置き換え、ANDで累積）
3. 2種類の検索を合わせる
   - 分類＋キーワード：3軸の用語と固有名詞
   - ベクトル（段階2）：質問を埋め込み、`wp_nfa_chunks` から上位を取り出す
4. 結果（新しい順に5件）、すべて見るリンク、次に絞れる軸、0件時に外す条件、左パネルの件数を返す

`POST /wp-json/nfa/v1/answer`

- 入力：質問＋ `/search` が返した上位の断片ID

- Claude Haiku 5.5 が、渡された断片だけを根拠に短い案内文を作り、出典のIDを付けて返す（段階3）
- 根拠がないときは答えず、外す条件や近い選択肢を提案する

返すデータの形

`/search` の応答は `cond`（いまの条件）、`total`、`say`（AIの返事）、`results`（日付・タイトル・3軸・URL）、`more`、`next`、`rescue`、`facets` です。担当委員会は応答に含めません。

11. フロントエンド

探索画面はプラグインのショートコード `[nfa_ai_search]` として提供し、ワイヤー `search.html` の画面と会話の流れを移植しました。テーマは固定ページにそれを置き、ページ見出しやヘッダーを担当します。

ワイヤーの要素	実装
左：条件パネル（件数、0件は押せない）	`/search` の `facets` を描画
右：いまの条件・該当件数・パネルの表示切り替え	`cond` と `total` を描画
会話（利用者の吹き出し、AIの返事、結果5件、さらに絞り込むなら、レスキュー）	`say` / `results` / `more` / `next` / `rescue` を描画
下部固定の入力欄、入口の例示	画面側で描画。TOPから `?q=` 付きで来たらすぐ検索

- ヘッダーの検索ボックスは `nfa_ai_search_form()`（入力内容を付けて探索画面へ移動）
- 記事ページは `nfa_ai_search_the_axes()`（3軸のチップ）と `nfa_ai_search_source_notice()`（根拠の表示）
- 「すべて見る」の行き先はタクソノミーで絞った一覧（`/?nfa_target=…&nfa_purpose=…`）。テーマの `archive.php` で表示する
- 固定ヘッダーの高さは CSS 変数 `--nfa-ais-sticky-top` で合わせる
- APIが失敗したときは、画面に「うまくつながりませんでした」と出し、条件はそのまま残す
- REST APIの呼び出しには nonce を付け、ページキャッシュで古くなったら自動で取り直す

12. 処理時間の見込み

検索結果は約1〜2秒で表示され、案内文はその後1〜3秒で追加されます。PHPの実行時間上限は30秒以上あれば十分です。

工程	API	時間の目安
① 質問を検索条件に変換（Haiku）	`/search`	0.5〜1.5秒
② 質問のベクトル化＋類似検索＋分類検索	`/search`	0.2〜0.5秒（MySQL 8.0 は0.3〜0.8秒）
③ 出典付きの案内文（Haiku）	`/answer`	1〜3秒

①と②のうち、ベクトル化は①と同時に実行できるため、`/search` は1〜2秒を目標にします。同じ質問は一時保存した結果を返すため、ほぼ即座に表示されます。

13. セキュリティ・コスト管理

APIキーはブラウザに一切出さず、悪用や費用の暴走は「回数制限・一時保存・月額上限」の3段で止めます。

対策	内容
APIキーの管理	`wp-config.php` に定数として置き、Gitには入れない
不正な呼び出しの防止	REST APIにnonceを必須にし、入力は最大200文字程度に制限
回数制限	IPアドレスごとに1分あたりの回数を制限（transientで計数）
結果の一時保存	質問を正規化してキーにし、結果を数時間保存。記事更新時に破棄
月額上限	Anthropicの管理画面で上限を設定。プラグイン側でも日次の利用量を記録し、しきい値を超えたらAIを止める
障害時の動作	AIが使えないときは、分類＋キーワードだけの検索に自動で切り替える
指示の乗っ取り対策	利用者の入力は「質問」としてのみ扱い、案内文は渡した断片の範囲だけで作らせる

お問い合わせ内容などの個人情報を検索対象から外す仕組みは、9章「検索対象と除外ルール」にまとめています。

14. 既存データ整備

既存の約5,000記事への3軸の付与は、AIで自動分類してから人が確認します。手作業だけでは現実的でないためです。

1. WordPressの記事を書き出す（WP-CLIまたはCSV）
2. Claude Haiku 5.5 のBatch API（通常の半額）で、各記事に3軸の候補と確信度を付ける
3. 確信度の低い記事だけを一覧にして、人が確認・修正する
4. 確定した分類をWordPressに一括で登録する

AI費用は数百円程度です。今後の新しい記事は、投稿画面で編集者が3軸を選んで公開します（AIによる候補の提示も可能）。PDF 300件は、どの記事に添付されているかを確認し、親記事の分類を引き継ぎます。

15. 運用

公開後は「0件だった質問」を定期的に見て、辞書と分類を育てていく運用にします。

作業	頻度	担当（案）
0件・聞き返しが多い質問を確認し、言い換え辞書に追加	月1回	協会事務局
API利用料と上限の確認	月1回	制作・保守担当
取り込みエラー（読めないPDFなど）の確認	月1回	制作・保守担当
新しい記事への3軸の付与	投稿のたび	各委員会の投稿者

- 検索ログは個人を特定できる情報（IPアドレスなど）を含めず、質問文と件数だけを保存する。質問文中のメールアドレスと電話番号は伏せ字にし、保存期間は90日とする
- プライバシーポリシーに、検索に入力された文章を外部のAIサービスで処理する旨を追記する
- 検索欄の近くに「個人情報は入力しないでください」と表示する